AI Accelerator Architecture Case Study

Three Most Challenging Technical Problems Solved

Project Overview

This project involved designing and analyzing an INT8 systolic-array accelerator on a Zynq SoC platform, targeting MobileNetV2 workloads in a full HW-SW co-design environment.

Although the 8x4 processing element array achieved up to 99% utilization at 100 MHz (3.2 GMAC/s peak), the overall end-to-end system speedup remained unexpectedly low.

Resolving this discrepancy led to three major technical challenges:

(1) Redefining performance measurement,
(2) Diagnosing structural memory bottlenecks using roofline analysis,
(3) Understanding how precision choices reshape system architecture

Problem 1

Why Did 99% PE Utilization Produce Only 2x System Speedup?

1. The Paradox

The accelerator achieved

- Average PE Utilization: 88.34%
- Peak Utilization: 99%
- Theoretical Peak: 3.2 GMAC/s

Yet the measured end-to-end speedup was only 2.04x

This contradiction indicated that compute saturation did not translate into system-level performance.

2. Measurement Redefinition

Initially, the Execute phase was treated as “pure compute”
However, further inspection revealed it included:

- DMA waiting
- Control overhead
- Internal stalls

The performance metric itself was incorrectly defined.

To isolate the true bottleneck, I redesigned the measurement infrastructure

- PL cycle counters separating Compute / DMA / Control
- Host runtime decomposition (MMIO / Layout / DMA wait)
- Introduction of a structural metric: PL Active Ratio

3. Quantitative Breakdown

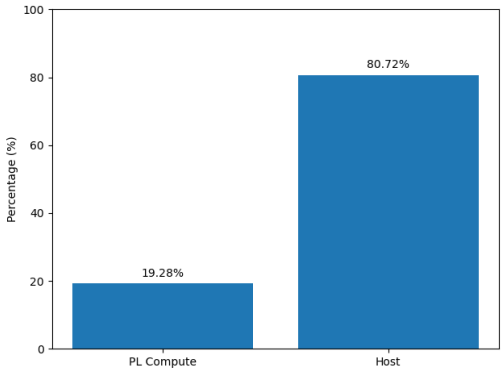

Figure 1. Runtime decomposition (System-Level)

PL compute occupies only 18.93% of total runtime.

Despite high utilization, the PL was active less than 20% of execution time.

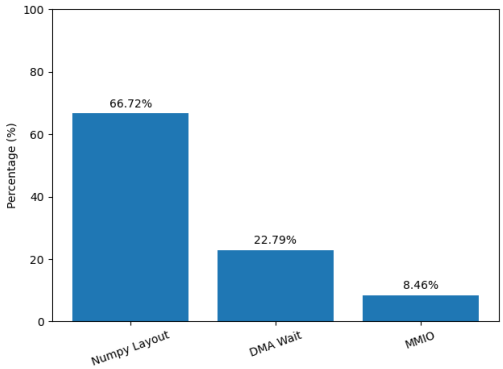

Figure 2. Host runtime breakdown

Over 66% of host runtime was spent on memory layout transformation.

4. Structural Interpretation

The refined measurements showed:

- Pure Compute Speedup: 36.85x
- End-to-End Speedup: 2.04x
- PL Active Ratio: 18.93%

The bottleneck was not computation capacity.

It was orchestration overhead and data movement structure.

High arithmetic density does not guarantee system throughput when the data pipeline is inefficient.

Key Insight

High utilization is a local metric.
System throughput is governed by data movement structure.

This fundamentally changed my interpretation of performance metrics in heterogeneous systems.

Problem 2

Why Did the Accelerator Remain Memory-Bound Even Inside the PL?

1. Hypothesis

Aside from host overhead (Problem 1),
I expected the PL to transition into a compute-bound regime.

It did not.

This suggested that the bottleneck existed inside the PL architecture itself.

2. Roofline Model Construction

To quantify the bottleneck, I constructed a roofline model using measured effective bandwidth.

Peak Compute:

- 32 MAC/cycle
- 100 MHz
- 3.2 GMAC/s

Measured effective bandwidth (via DMA cycle counters)

- 0.11 ~ 0.30 GB/s

This yielded a ridge-point arithmetic intensity:

AI_{ridge} = 19 MAC / Byte

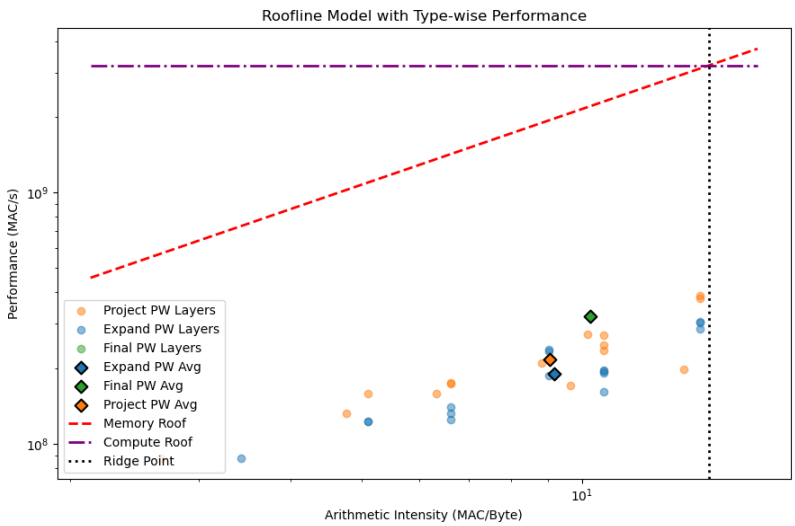

Figure 3. Roofline Model with measured effective bandwidth

Layer-wise arithmetic intensity:

- 2-15 MAC/Byte

All layers fell to the left of the ridge point.

The design could not enter the compute-bound region

3. Model Validation via Correlation

In the memory-bound regions:

Performance ≈ AI x Bandwidth

To validate this, I performed correlation analysis across layers.

Variable	Correlation with Speedup
AI	0.76
Bandwidth	0.49
AI × Bandwidth	0.83

Table 1. (AI x Effective BW) vs End-to-End Speedup correlation

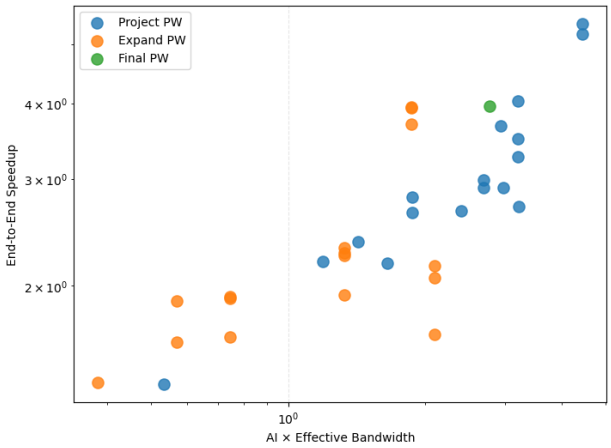

Figure 4. (AI x Effective BW) vs End-to-End Speedup

The near-linear trend confirmed that output throughput variation was dominated by memory behavior.

4. Structural Causes

The memory-bound behavior was not incidental.

It was structurally embedded in the architecture:

- INT8 data transferred in 32-bit units (no packing)
- Tile-based DMA repetition
- On-chip SRAM depth constraints forcing IC/OC partitioning
- Limited PSUM reuse

Arithmetic Intensity was not algorithm-defined. It was implementation-defined

Key Insight

The accelerator was not temporarily memory-bound.
it was structurally memory-bound by design.
Compute scaling would not improve throughput without reducing memory-per-MAC.

Problem 3

Why Did Accumulator Precision Redefine System Throughput?

1. The Numerical Question

In INT8 x INT8 accumulation across the IC dimension,
accumulator bit-width became critical.

Theoretical worst-case:

- Max product: 16384
- IC up to 960
- Worst-case accumulation: 15,728,640

24-bit signed maximum:

- 8,388,607

Worst-case analysis suggested possible overflow.

2. Empirical Validation

To evaluate practical risk, I measured layer-wise PSUM maxima under QAT-trained MobileNetV2.

- Global maximum PSUM: 329,656
- 24-bit signed maximum: 8,388,607
- Utilization of dynamic range: ~3.9%

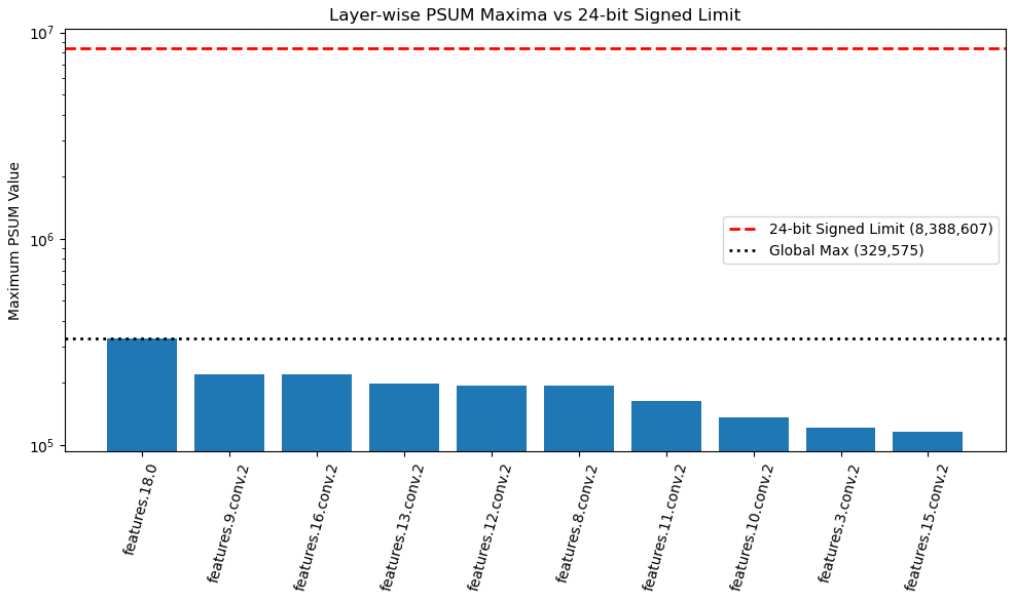

Figure 5. Layer-wise PSUM maxima vs 24-bit limit

Statistically, 24-bit accumulation was safe.

Worst-case theoretical bounds were overly conservative for real distributions.

3. Architectural Consequence

However, the deeper issue was not overflow.

It was architecture.

Increasing accumulator width triggered cascading structural effects:

Accumulator width ↑
→ PSUM SRAM depth ↓
→ Tile_OC ↓
→ Repeat ↑
→ Effective Bandwidth ↓
→ Throughput ↓
Precision directly altered memory-per-MAC.
It reshaped the tiling strategy and memory hierarchy.

4. Engineering Trade-off
In a research context, 24-bit was sufficient.
In a production environment, one must consider:

- Input distribution shifts
- Calibration dataset limitations
- Safety margin requirements

The design choice became a trade-off:

Stability vs Resource Usage vs Throughput

Key Insight

Precision is an architectural variable.

It influences memory per MAC, which ultimately governs system throughput.

Final Synthesis

Converging Toward a Structural Constraint

These three challenges were not isolated.

They were manifestations of a single structural constraint:

Memory hierarchy and memory-per-MAC

1. Problem Convergence

- Problem 1 showed that compute utilization did not determine system throughput
- Problem 2 demonstrated that arithmetic intensity and effective bandwidth constrained PL performance.
- Problem 3 revealed that precision choice reshaped tiling and SRAM allocation

2. Structural Reframing

Compute parallelism was not the limiting factor.
DMA was not merely an interface layer.
Precision was not a local arithmetic decision.

Each was coupled through memory hierarchy.

The accelerator's performance was governed not by MAC parallelism, but by how efficiently memory supported those MACS.

3. Perspective Shift

This project fundamentally shifted my design perspective:

- From increasing MAC parallelism $\rightarrow$ reducing memory-per-MAC
- From maximizing utilization $\rightarrow$ restructuring dataflow
- From preventing overflow $\rightarrow$ balancing architectural trade-offs

Performance does not emerge from arithmetic units alone.

It emerges from system structure.